Transformers & Pre-trained Models

Building on attention to modern NLP

Deep Learning Course

1. Introduction
2. Transformer Architecture
3. Masked Attention
4. Pre-trained Models: The Game Changer
5. BERT: Encoder-Only Pre-training
6. GPT: Decoder-Only Pre-training
7. BERT vs. GPT
8. Transfer Learning: Fine-tuning

Section 1

Introduction

From Attention to Transformers

Class 4: Attention mechanism (one building block)

This class: Transformer architecture (complete system) - Encoder: processes input in both directions - Decoder: generates output step-by-step - Pre-trained models: BERT, GPT, etc.

```
Transformer = Encoder + Decoder
            = Multi-head attention + Feed-forward
            + Positional encoding + Layer normalization
```

Section 2

Transformer Architecture

The Full Picture

```
Input: "Hello world"

Embedding + Positional Encoding
    ↓
Encoder Block (repeat 12x)
  ├─ Multi-Head Attention
  ├─ Add & Norm
  ├─ Feed-Forward Network
  └─ Add & Norm
    ↓
Decoder Block (repeat 12x, if generating)
  ├─ Masked Multi-Head Attention (past only)
  ├─ Cross Attention (to encoder output)
  ├─ Feed-Forward
    ↓
Output Layer: Softmax (for text generation)
```

Encoder vs. Decoder

Component	Role	Examples
Encoder	Understand input	BERT, classification
Decoder	Generate output	GPT, translation
Both	Sequence-to-sequence	Machine translation, summarization

Section 3

Masked Attention

Why Masking Matters

Encoder: Can see entire sequence (bidirectional)

```
"The movie was great"
  ↓ (self-attention)
Each word attends to: "The", "movie", "was", "great"
```

Decoder: Cannot see future (must generate left-to-right)

```
Generating "great" after "The movie was"
  ↓ (masked attention)
Can only attend to: "The", "movie", "was" (not "great"!)
Prevents cheating: model shouldn't know future
```

Implementation: Set future position scores to $-\infty$ before softmax

Section 4

Pre-trained Models: The Game Changer

Why Pre-training Works

Training from scratch: - Expensive (compute, time, data) - Requires task-specific labels - Limited by labeled data

Pre-training: 1. Train on huge unlabeled corpus (Wikipedia, Common Crawl, etc.) 2. Learn general language understanding 3. Fine-tune on specific task (requires few labeled examples)

Analogy: - From scratch: teach someone to play tennis from fundamentals - Pre-training: teach someone fluent in sports, then tennis specifics

Section 5

BERT: Encoder-Only Pre-training

What is BERT?

BERT = Bidirectional Encoder Representations from Transformers

Architecture: - 12-24 transformer encoder blocks - 110M-340M parameters - Trained on 3.3B words (Wikipedia + Books)

Pre-training objectives:

1. **Masked Language Model (MLM)**: Predict masked words

```
Input:  "The [MASK] was very good"
Predict: "The movie was very good"
         (15% of words randomly masked)
```

2. **Next Sentence Prediction (NSP)**: Is sentence B next after A?

```
Sent A: "The movie was great"
Sent B: "I loved every moment"
Label: YES
```

Section 6

GPT: Decoder-Only Pre-training

What is GPT?

GPT = Generative Pre-trained Transformer

Architecture: - Decoder-only (can't see future) - 12-96 transformer blocks - Billions of parameters (GPT-3 = 175B) - Trained on 300B+ tokens

Pre-training objective: Next token prediction

```
Input:  "The movie was very"
Predict: "good" (next token)

Input:  "The movie was very good"
Predict: "." (next token)
```

Key difference: Generate text, not understand it

Section 7

BERT vs. GPT

Comparison

Aspect	BERT	GPT
Architecture	Encoder (bidirectional)	Decoder (unidirectional)
Pre-train	Masked language model	Next token prediction
Best for	Classification, understanding	Generation, few-shot
Fine-tuning	Add classification head	In-context learning
Examples	Sentiment, NER, QA	Chatbot, text completion

Section 8

Transfer Learning: Fine-tuning

From Pre-trained to Task-Specific

Step 1: Load pre-trained weights

```
from transformers import BertForSequenceClassification

model = BertForSequenceClassification.from_pretrained(
    'bert-base-uncased',  # Pre-trained BERT
    num_labels=2  # Binary classification
)
```

Step 2: Add task-specific head (output layer)

Step 3: Fine-tune on your task

```
optimizer = keras.optimizers.Adam(learning_rate=2e-5)
model.compile(optimizer=optimizer, loss='sparse_categorical_crossentropy')
model.fit(x_train, y_train, epochs=3, batch_size=32)
```

Section 9

HuggingFace Transformers Library

The Standard Tool

HuggingFace = open-source library for transformers

```
pip install transformers torch  # or tensorflow
```

Key features: - 50,000+ pre-trained models - Easy API (load, tokenize, predict) - Auto-downloading weights - Community-driven

Section 10

Practical Example: Setup

Sentiment Classification with BERT

Task: Classify movie reviews (IMDb)

Approach: 1. Load pre-trained DistilBERT (smaller BERT) 2. Fine-tune on sentiment classification 3. Evaluate on test set

Why DistilBERT? - 40% smaller, 60% faster than BERT - 97% of BERT's accuracy - Fits on consumer GPUs

Section 11

Practical Example: Implementation

Tokenization and Model

```python
from transformers import AutoTokenizer, TFAutoModelForSequenceClassification
import tensorflow as tf
from tensorflow import keras

# Load tokenizer and model
model_name = 'distilbert-base-uncased'
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = TFAutoModelForSequenceClassification.from_pretrained(
    model_name,
    num_labels=2
)

# Prepare data
texts = ["I loved this movie!", "This was terrible"]
labels = [1, 0]  # positive, negative
```

Section 12

Practical Example: Training

Fine-tuning BERT

```
# Compile
optimizer = keras.optimizers.Adam(learning_rate=2e-5)
loss = keras.losses.SparseCategoricalCrossentropy(from_logits=True)

model.compile(
    optimizer=optimizer,
    loss=loss,
    metrics=['accuracy']
)

# Train
history = model.fit(dataset, epochs=3)

# Predict
test_texts = ["Great movie!", "I hated it"]
test_encoding = tokenizer(
```

Section 13

Scaling Transformers

Why They Work So Well

Three reasons:

1. **Parallelization**
   - Unlike RNNs, compute all positions simultaneously
   - GPUs/TPUs excel at parallel matrix operations
   - 100x faster than RNNs
2. **Scaling Laws**
   - Transformer performance improves reliably with more data/parameters
   - GPT-3 (175B params) » BERT (110M params)
   - Predictable improvements
3. **Transfer Learning**
   - Pre-trained on billions of tokens
   - Fine-tune with minimal labeled data
   - Start from strong baseline

Section 14

Common Use Cases

Beyond Classification

Task	Architecture	Example
Classification	BERT + head	Sentiment, spam detection
Generation	GPT-like	Chatbot, summarization
QA	BERT + span extraction	Find answer in document
Named Entity	BERT + token classification	Extract person, location
Translation	Encoder-decoder	Seq2seq translation

Section 15

Practical: Question Answering

SQuAD-style QA

```python
from transformers import pipeline

# One-liner with HuggingFace pipeline
qa_pipeline = pipeline('question-answering',
                       model='distilbert-base-uncased-distilled-squad')

context = """
The Eiffel Tower is a wrought iron lattice tower on the Champ de Mars
in Paris. It was named after the engineer Gustave Eiffel.
"""

question = "Who was the Eiffel Tower named after?"

result = qa_pipeline(question=question, context=context)
print(f"Answer: {result['answer']}")
print(f"Confidence: {result['score']:.2%}")
```

Section 16

Text Generation

With GPT-2

```
from transformers import pipeline

# Load GPT-2
text_gen = pipeline('text-generation',
                    model='gpt2')

prompt = "The future of AI is"

generated = text_gen(
    prompt,
    max_length=50,
    num_return_sequences=3,
    temperature=0.7,
    top_p=0.9
)
```

Section 17

Inference & Deployment

Making Predictions

```python
# Simple inference
def classify_sentiment(text):
    encodings = tokenizer(
        text,
        padding='max_length',
        max_length=128,
        truncation=True,
        return_tensors='tf'
    )
    logits = model(encodings).logits
    probs = tf.nn.softmax(logits)
    label = tf.argmax(probs).numpy()
    confidence = float(tf.reduce_max(probs))
    return label, confidence

# Batch inference (more efficient)
```

Section 18

Performance Considerations

Speed vs. Accuracy

Model	Size	Speed	Accuracy	Use Case
DistilBERT	66M	Fast	97%	Real-time
BERT	110M	Slow	100%	Batch
RoBERTa	355M	Slower	102%	Max accuracy
ALBERT	12M	Very Fast	95%	Mobile

Recommendation: Start with DistilBERT, scale if needed

Section 19

Limitations & Bias

Important Caveats

What transformers can't do: - Reason (follow logic chains) - Count accurately (struggle with "how many") - Use external knowledge (unless trained on it) - Explain decisions (black box)

Bias concerns: - Training data encodes societal biases - Models amplify existing prejudices - Important for sensitive applications

Best practice: Audit models for bias before deployment

Section 20

Key Concepts Summary

What We've Covered

- Transformer architecture (encoder + decoder)
- Pre-trained models (BERT, GPT)
- Transfer learning and fine-tuning
- HuggingFace library and pipelines
- Common use cases (classification, generation, QA)
- Performance trade-offs
- Limitations and ethical considerations

The Big Picture

Transformers revolutionized NLP by: - Parallelizable (fast training) - Pre-trainable (leverage unlabeled data) - Transfer-learnable (fast adaptation) - Scalable (bigger = better)

Section 21

Beyond Text

Recent Extensions

Transformers now work on:

Domain	Models	Examples
Vision	ViT, DeiT	Image classification, detection
Audio	Wav2Vec, HuBERT	Speech recognition
Multimodal	CLIP, DALL-E	Image-text, generation
Graphs	GraphTransformer	Node prediction

Message: Attention + scaling works everywhere!

Section 22

Practical: Production Considerations

Deploying Models

```python
# Export for production
model.save_pretrained('my_model')
tokenizer.save_pretrained('my_model')

# Later, load in production
from transformers import AutoTokenizer, AutoModel
tokenizer = AutoTokenizer.from_pretrained('my_model')
model = AutoModel.from_pretrained('my_model')

# Optimize with ONNX
from transformers.onnx import convert_pytorch_to_onnx
convert_pytorch_to_onnx(...)

# Use in web service (FastAPI example)
from fastapi import FastAPI
app = FastAPI()
```

Section 23

Exercises for Class 5

Hands-On Tasks

1. **Fine-tune for custom task**: Use your own sentiment data. Compare to pre-trained directly.
2. **Model comparison**: Test BERT vs. DistilBERT vs. RoBERTa. Speed-accuracy tradeoff?
3. **Few-shot learning**: Fine-tune on 10, 50, 100 labeled examples. How much data needed?
4. **Transfer learning**: Train on IMDb, test on different review dataset. Generalization?
5. **Generation**: Fine-tune GPT-2 on domain-specific text (recipes, code). Check quality.
6. **Interpretability**: Use attention visualization. Which words drive predictions?
7. **Deployment**: Save model, load in separate script, create batch inference.

Section 24

Course Summary

From Neural Networks to Transformers

Class	Topic	Key Idea
1	Neural Networks	Gradient-based learning
2	Embeddings	Words as vectors
3	RNN/LSTM/CNN	Sequence processing
4	Attention	Parallel context
5	Transformers	Scalable pre-training

Trajectory: Architectures got better, training got faster, models got bigger

Section 25

Future Directions

What's Next in NLP?

- **Larger models** (1T+ parameters)
- **Multimodal** (text + image + audio)
- **Efficient** (mobile, edge)
- **Interpretable** (explainability)
- **Multilingual** (low-resource languages)
- **Reasoning** (beyond pattern matching)

Section 26

Final Recommendations

Best Practices

1. **Start simple**: Averaged embeddings before LSTM
2. **Use pre-trained**: Don't train from scratch
3. **HuggingFace first**: Battle-tested, documented
4. **Monitor bias**: Important for production
5. **Ablate components**: Understand what matters
6. **Iterate rapidly**: Small experiments, learn fast

Remember: Most recent papers are not needed for production. Solid fundamentals + good engineering > cutting-edge complexity.

Section 27

Questions & Discussion

Questions & Discussion

Thank you!

Key takeaway: Transformers combine attention, scaling, and pre-training to create powerful NLP models. Master these fundamentals, use libraries like HuggingFace, and you can tackle almost any text task.

Good luck with your projects!